TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie

Modélisation UML

Chapitre 6: Le Diagramme de Collaboration

Auteur :

Mme Ines AGREBI

UNIFIED MODELING LANGUAGE™

Définition d’un diagramme de collaboration

❖**Définition:**

- Le diagramme de collaboration fait partie des vues dynamiques du système d’information.

- Ce type de diagramme montre les interactions et les liens entre objets (instances de classes et acteurs). Il s'intéresse à la structure de collaboration entre objets (séquencement, itération, concurrence, etc.).

- Il permet de représenter le contexte d'une interaction, car on peut y préciser les états des objets qui interagissent.

- Les diagrammes de collaboration montrent les interactions entre les objets à travers la représentation chronologique d'envois de messages, mais le temps n'est pas représenté implicitement. La chronologie des interactions est indiquée par la numérotation de messages pour indiquer leur ordre d'envoi.

❖Notion de collaboration:

❑Principe de modélisation objet

- Les objets sont indépendants
- Chaque objet est responsable de certaines activités
- Pour réaliser une activité, il faut le concours collaboratif d'objets
- Les objets collaborent via des interactions

➔ Système basé sur l'individu « objet »

❑Collaboration

- Modélise une activité réalisée par un ensemble d'objets
- Peut représenter un scénario d'un cas d'utilisation du point de vue du fonctionnement interne du système

❖Objectifs:

- Ce type de diagramme montre des interactions entre objets (instances de classes et acteurs). Il s'intéresse à la structure de collaboration entre objets (séquencement, itération, concurrence, etc.)
- Il permet de représenter le contexte d'une interaction, car on peut y préciser les états des objets qui interagissent.
- Ce diagramme est équivalent ***au diagramme de séquences***. Cependant, l'aspect temporel n'apparaît pas, mais l'aspect chronologique est présent.
- Le diagramme de collaboration représente des éléments réalisant un objectif particulier (une fonctionnalité du système). Il peut être utilisé pour illustrer l'exécution d'une opération ou d'un cas d'utilisation, ou simplement un scénario d'interactions dans le système. Il permet de concevoir un exemple d'interactions entre objets.

❖Éléments de base:

- Un tel diagramme est formé d'un sous-ensemble ***du diagramme de classes*** et les messages échangés entre objets à travers les liens.

- Pour représenter l'aspect chronologique (ordre d'envoi des messages), il est important de numéroter les messages.

- Le sens des messages nous permet de déterminer l'envoyeur et le receveur.

Composants du diagramme:

Les diagrammes de collaboration comportent les éléments suivants:

- Les objets (instances de classe)
- Les liens entre objets (comme dans le diagramme de classe)
- Les interactions entre objets sous forme d'une suite de messages.
- Les acteurs

❖ Objet:

- Un objet est une instance d'une classe. Il peut-être persistant ou transitoire : un objet persistant est un objet qui continue d'exister une fois que le processus qui l'a créé est terminé, un objet transitoire est un objet qui cesse d'exister une fois que le processus qui l'a créé est terminé.

- Les objets sont obtenus à partir des classes. Chaque objet est décrit par : Nom-objet :nom-classe.

- Un multiple définit un jeu d'instances. Il s'agit d'une représentation graphique d'un objet qui présente plusieurs instances, il ne peut en revanche avoir qu'un seul ensemble d'attributs et ce, même s'il représente plusieurs instances. Un objet peut communiquer avec un autre objet qui est un multiple. Cette fonctionnalité est principalement utilisée dans le diagramme de collaboration mais peut également être utilisée dans le diagramme d'objets.

Exemple:

Un employé gère une liste de documents : c'est la liste qui représente l'objet multiple.

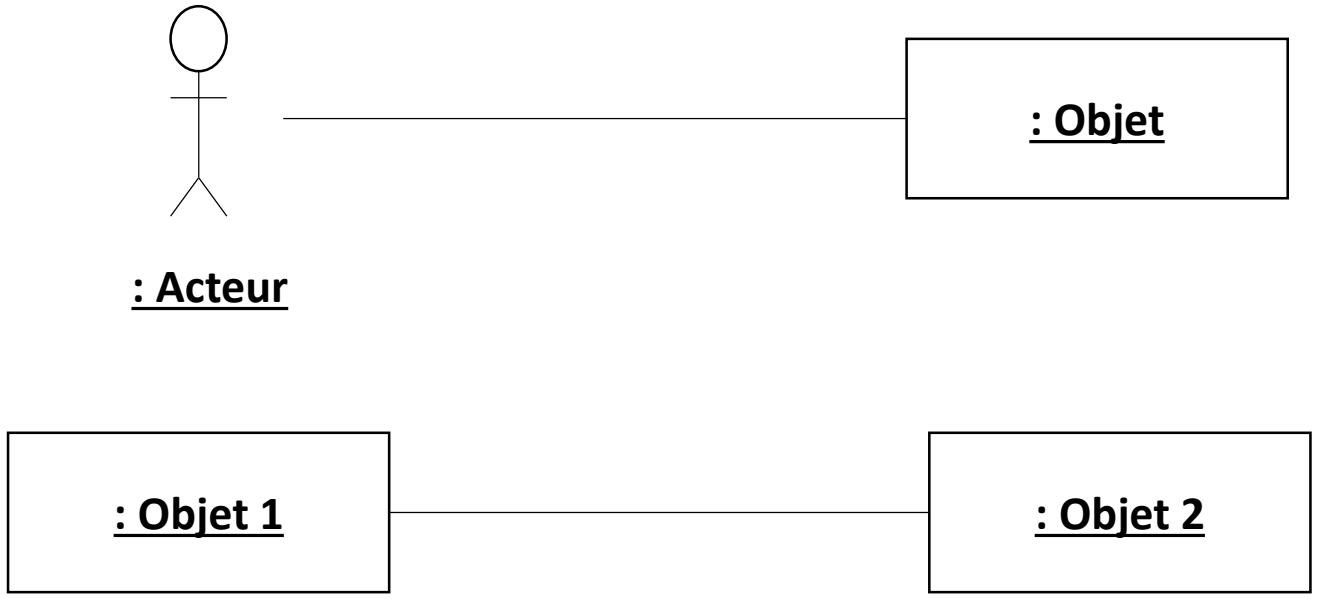
❖ Liens:

- Un lien entre objets représente une connexion entre deux objets, il permet de mettre en exergue la collaboration entre objets, d'ou le nom de diagramme de collaboration. Il est représenté sous forme d'un trait plein entre:

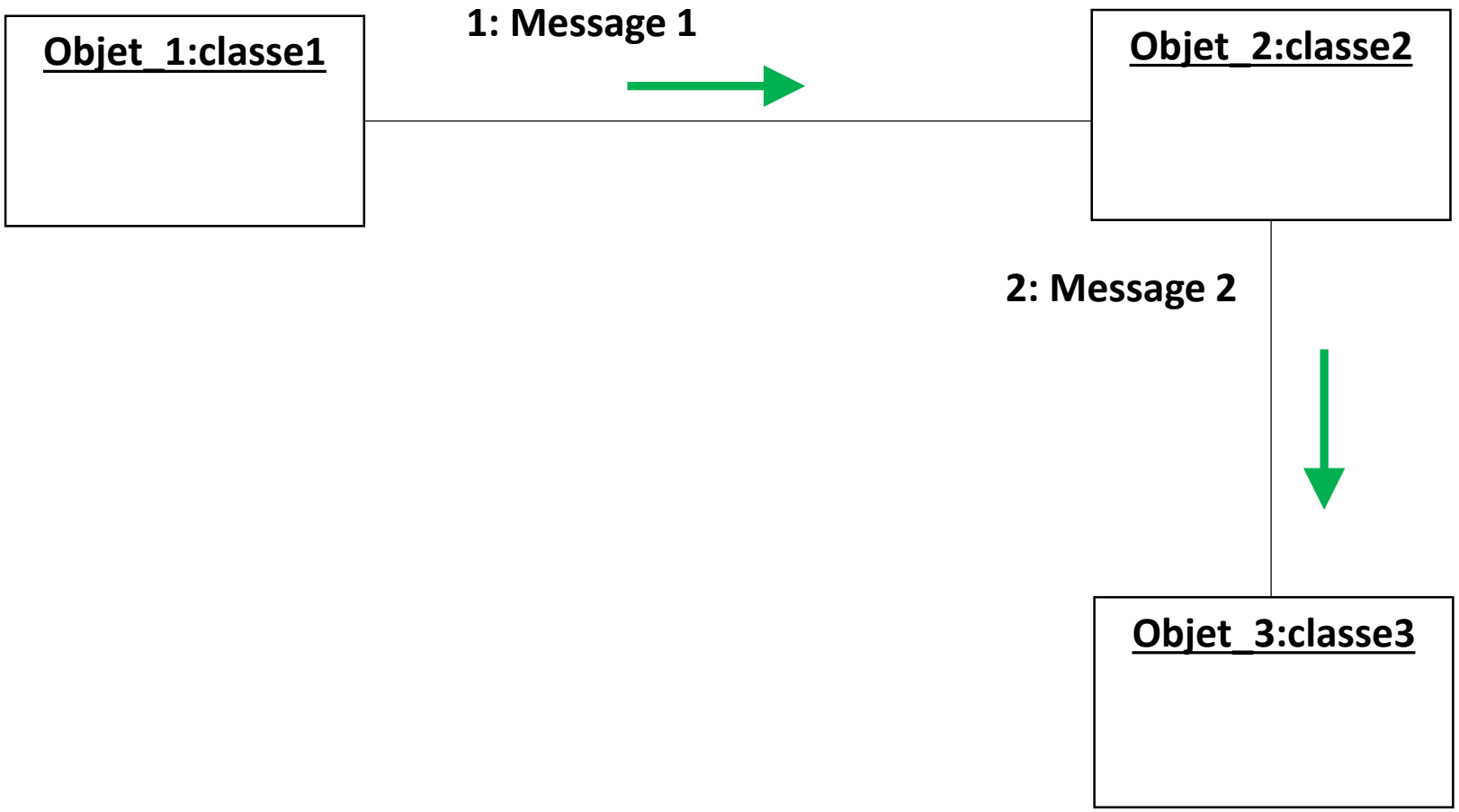
- Deux objets
- Un objet et un acteur (ou vice-versa)

- Un lien entre objets peut être une instance d'association entre classes.

Exemple:



Composants du diagramme de collaboration:

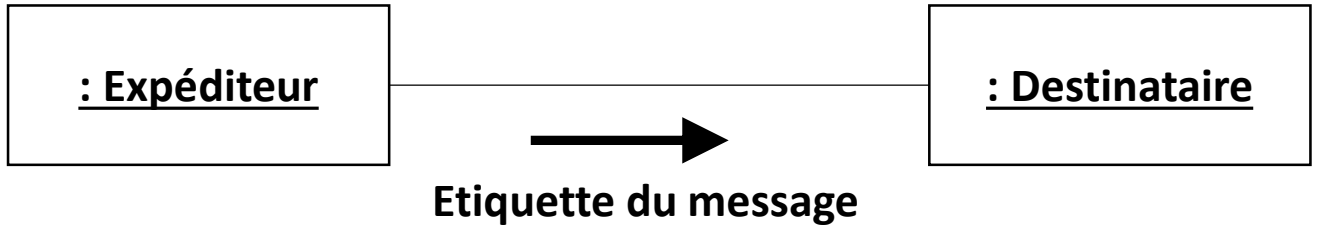


❖Acteurs:

- La définition d'un acteur dans le contexte d'un diagramme de collaboration est la même que celle en vigueur dans les diagrammes de cas d'utilisation ou de séquence ; elle spécifie un utilisateur externe, ou un ensemble d'utilisateurs liés qui interagissent avec un système.

❖Message, synchronisation des messages:

- UML permet de spécifier de manière très précise l'ordre et les conditions d'envoi des messages sur un diagramme dynamique.
- Un message représente une demande faite par l’objet émetteur à l’objet destinataire.
- Le destinataire doit pouvoir comprendre le message(méthode de l’objet destinataire, nécessité d’une association entre les classes correspondantes des objets).



❖Message, synchronisation des messages:

- Dans un diagramme de collaboration il est possible de spécifier de manière très précise l'ordre et les conditions d'envoie des messages. Pour chaque message, il est possible d'indiquer :

➢Son numéro d'ordre: qui indique le rang du message, c'est-à-dire son numéro d'ordre par rapport aux autres messages. Les messages sont numérotés à l'aide de chiffres séparés par des points. Ainsi, il est possible de représenter le niveau d'emboîtement des messages et leur précédence.

Exemple: l'envoi du message 1.3.5 suit immédiatement celui du message 1.3.4 et ces deux messages font partie du flot (de la famille de messages) 1.3.

- Pour représenter l'envoi simultané de deux messages, il suffit de les indexer par une lettre.

Exemple: l'envoi des messages 1.3.a et 1.3.b est simultané.

- La syntaxe minimale d’un message est la suivante :
- *séquence : message()*
- **séquence** : numéro d'ordre du message par rapport aux autres messages.
- **message** : nom du message.

Exemple: *2 : préparer_commande()*

- Cette syntaxe peut avoir des éléments optionnels :

- séquence : message([par])

- : paramètres (optionnels) du message.

Exemple: *2 : préparer_commande(liste_produit)*

- *[pré/] séq : msg([par])*
- **pré**: Indique que le message courant ne sera envoyé que lorsque tous ses prédécesseurs pré le seront aussi.

Exemple: *3/ 6 : ouvrir()*

- *[pré/] [[cond]] séq : msg([par])*
- **cond** : Permet de conditionner l'envoi du message, à l'aide d'une clause exprimée en langage naturel.

Exemple: *[commande prête] 8 : livrer(commande)*

- ✓ La liste des messages prédécesseurs, composée d'une liste de numéros d'ordre suivie de '/ ', la liste des prédécesseurs définit quels messages doivent être échangés avant que le message courant ne puisse être envoyé.

Exemple: numéros de séquence 1, 2, 4 avant 3 => '1,2,4/ 3'

- ✓Heure de début / Heure de fin, heures définies par l'utilisateur et utilisées pour définir des contraintes.

Exemple: [heure = midi] 1 : manger()

Ce message n'est envoyé que s'il est midi

✓Récurrence du message. Permet de spécifier l'envoi répétitif de messages en mode séquentiel (ou en parallèle, avec "||")

Exemple1: 1.3.6 * : ouvrir()

Ce message est envoyé de manière séquentielle un certain nombre de fois.

Exemple2: 3 / *||[i := 1..5] : fermer()

Représente l'envoi en parallèle de 5 messages. Ces messages ne seront envoyés qu'après l'envoi du message 3.

❖Autres exemples:

- 1 : bonjour()
  Ce message à pour numéro de séquence « 1 ».

- [heure = minuit] 1 : chercher()
  Ce message n'est envoyé que s'il est minuit

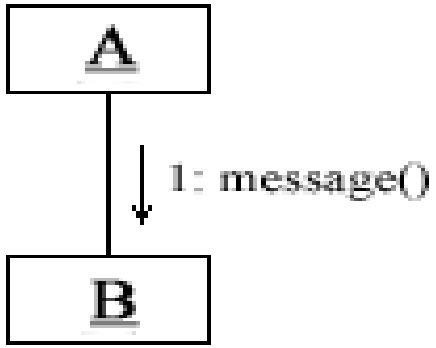
- 1.2.5 * : ouvrirfiche()
  Ce message est envoyé de manière séquentielle un certain nombre de fois

- 1.3,2.1 / [t < 10s] 2.5 : age := demanderAge(nom,prenom)
  Ce message (numéro 2.5) ne sera envoyé qu'après les messages 1.3 et 2.1, et que si « t < 10s ».

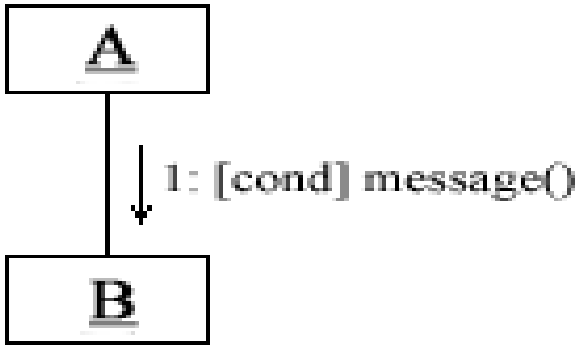
- 1.3 / [disk full] 1.7.a * : deleteTempFiles()
- 1.3 / [disk full] 1.7.b : reduceSwapFile(20%)

➔Ces messages ne seront envoyés qu'après l'envoi du message 1.3 et si la condition "disk full" est réalisée. Si cela est le cas, les messages 1.7.a et 1.7.b seront envoyés simultanément.

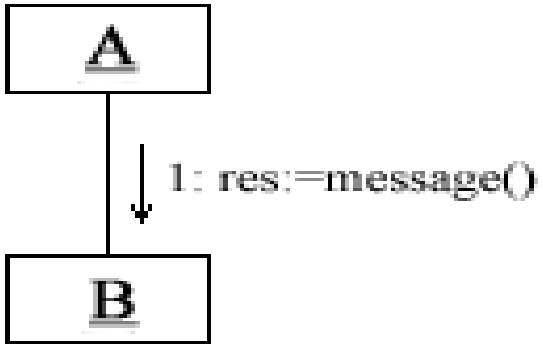
❖ Syntaxe des messages:



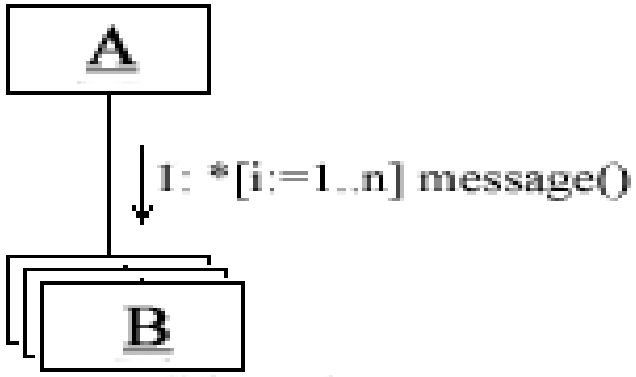
Message simple



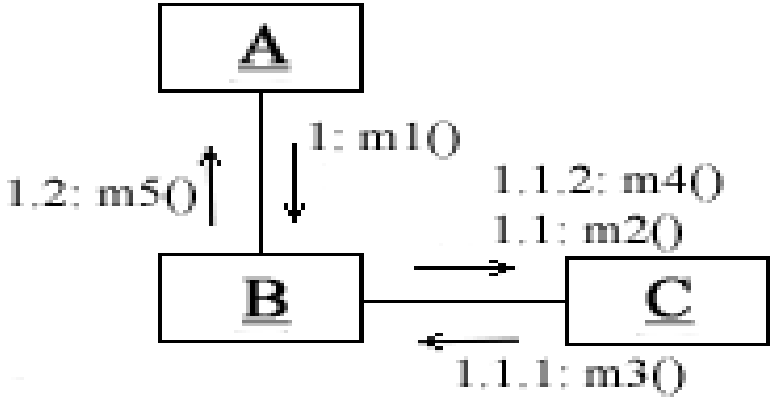
Message conditionnel



Message avec retour



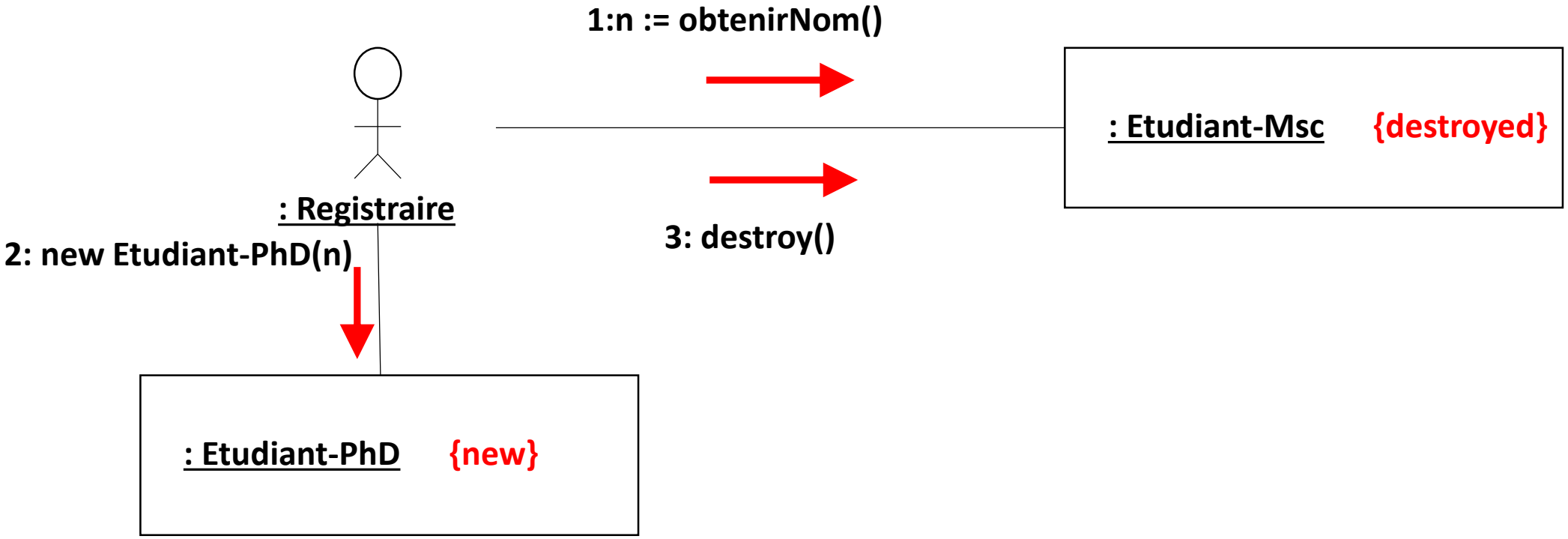
Itération



Raffinement

❖Création et destruction dynamique:

- Ajout d'une contrainte ({new} ou {destroyed}) après l'étiquette dans le rectangle représentant l'instance.
- Utilisation des messages de création et de destruction d'instance: new, destroy.



Exemple : Recrutement via un cabinet de recrutement

Diagramme de classe:

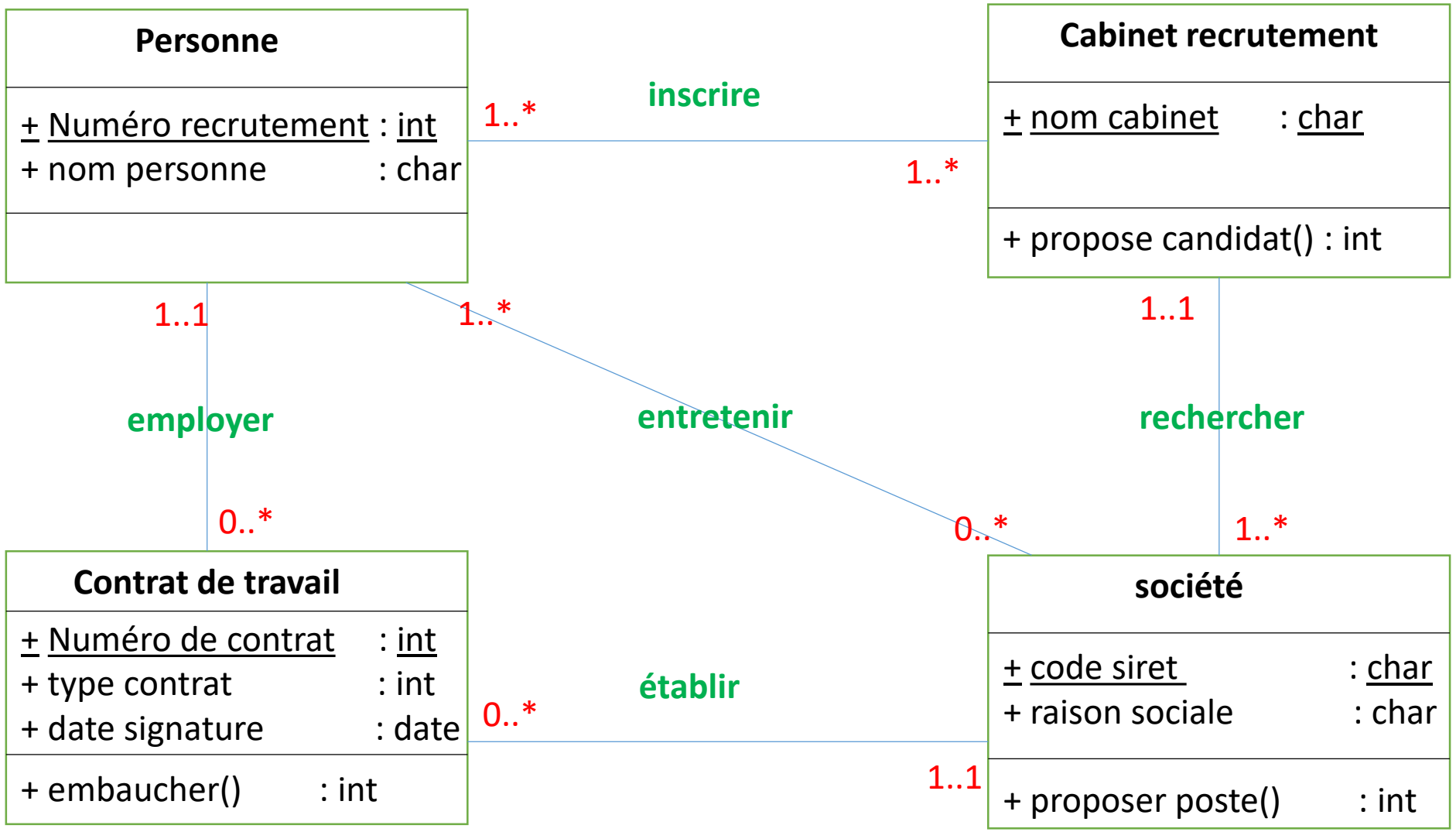
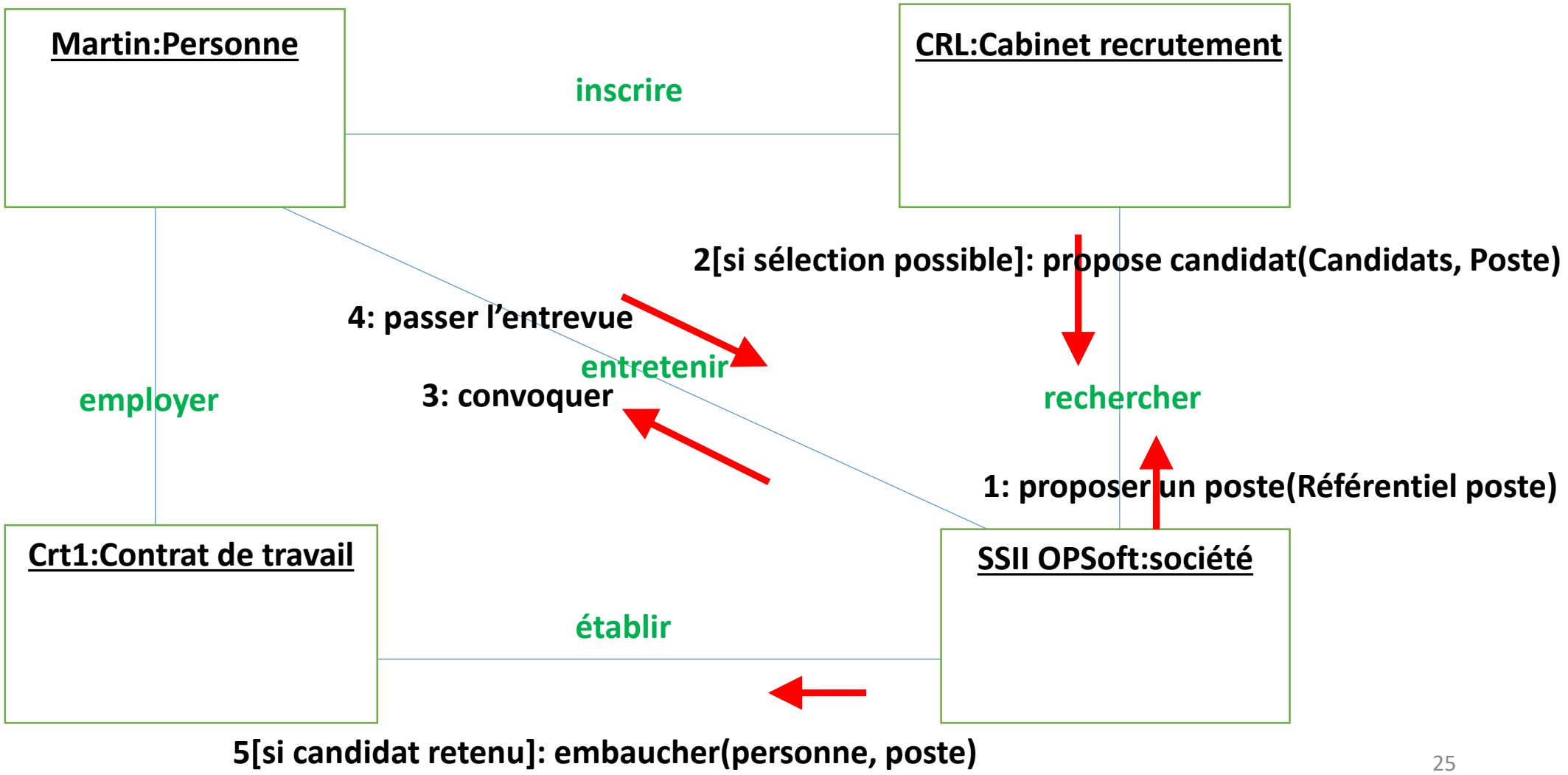


Diagramme de collaboration:



Exemple : Une autre façon de modéliser une collaboration

Diagramme de séquence:

- Une vue orientée temps versus orienté objet
- Plus facile à lire sur la vie des objets (Création, destruction, activité, parallélisme et déroulement du temps)
- Moins fine sur la composition des objets (Attributs)
- Le même formalisme que celui utilisé pour décrire l'interaction Acteur/Système de scénario.

Merci Pour Votre Attention